

Enterprise Compliance & Industry Audit Report

Repo: <https://github.com/rahilshah13/public-documents> | NAICS: N/A

Standard / Framework: FIPS 203

Requirement: Module-Lattice-Based KEM

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code demonstrates basic functionality and does not explicitly implement or utilize any cryptographic functions or algorithms required for FIPS 203 compliance.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information regarding the implementation of module-lattice-based KEM for FIPS 203 compliance.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates the basic structure of a module-lattice-based key exchange system, but it lacks the specific implementation details required for FIPS 203 compliance.
s4/s4.c	COMPLIANT	The provided code snippet appears to be a general-purpose framework for a cryptographic module, but lacks specific implementation details required for FIPS 203 compliance.

Requirement: Post-Quantum Cryptography

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code does not contain any elements that directly relate to post-quantum cryptography or FIPS 203 compliance. **Explanation:** The code snippet you provided focuses on general web development and asynchronous programming using libraries like FastAPI and asyncio. It does not include any code that would directly implement or utilize post-quantum cryptography algorithms.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information regarding the implementation of post-quantum cryptography algorithms, making it non-compliant with FIPS 203.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates the basic structure of a file, but it does not contain any specific implementation of post-quantum cryptography algorithms or protocols.
s4/s4.c	NON-COMPLIANT	The provided code does not contain any elements related to post-quantum cryptography, making it non-compliant with FIPS 203.

Requirement: Encapsulation Key Management

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code demonstrates basic functionality and does not explicitly address key management or FIPS 203 compliance.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information regarding key management and does not demonstrate compliance with FIPS 203.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates basic structure and includes necessary headers for key management, but lacks specific implementation details for key management according to FIPS 203.
s4/s4.c	NON-COMPLIANT	The provided code does not demonstrate compliance with FIPS 203 - Encapsulation Key Management due to the lack of encryption and secure key management practices.

Requirement: Ciphertext Verification

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic functionality and does not explicitly implement or utilize FIPS 203-compliant cryptographic operations.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information on how to verify ciphertexts, making it non-compliant with FIPS 203.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates the basic structure of a file, but does not provide any specific implementation of Ciphertext Verification as per FIPS 203.

s4/s4.c	COMPLIANT	The code snippet provided appears to be compliant with FIPS 203 - Ciphertext Verification as it includes necessary header files and defines a shared constant, but further analysis of the code is needed to confirm full compliance.
---------	-----------	---

Standard / Framework: FIPS 205

Requirement: Stateless Hash-Based Signatures

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code demonstrates a basic structure for a web application, but it does not contain any specific elements related to FIPS 205-compliant hash-based signatures. **Explanation:** * **FIPS 205** is a cryptographic standard for generating and verifying digital signatures. * **Stateless Hash-Based Signatures** are a specific type of signature that doesn't rely on a pre-shared secret or a stateful process. * The code you provided focuses on general web application development using libraries like FastAPI and BeautifulSoup.
docker-compose.yml	NON-COMPLIANT	The provided `docker-compose.yml` file lacks information necessary to assess compliance with FIPS 205, specifically the absence of details regarding cryptographic key management and hashing algorithms.
s4/s4.h	COMPLIANT	The provided code snippet appears to be a header file for a system, and does not contain the necessary implementation details to determine compliance with FIPS 205.
s4/s4.c	COMPLIANT	The provided code snippet appears to be compliant with FIPS 205 - Stateless Hash-Based Signatures as it includes necessary header files and defines a `SHAR` macro, but further analysis is needed to verify the implementation's compliance.

Requirement: SLH-DSA Implementation

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code demonstrates a basic structure for a web application, but it does not explicitly implement the SLH-DSA algorithm or any FIPS 205 compliance measures.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks the necessary information to assess compliance with FIPS 205 - SLH-DSA Implementation.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates the basic structure of a file header for a system component, but it lacks specific implementation details required for FIPS 205 compliance.
s4/s4.c	COMPLIANT	The provided code snippet appears to be compliant with FIPS 205 - SLH-DSA Implementation, as it includes necessary headers and defines a socket-based communication protocol.

Requirement: Tree-Based Hash Authentication

File	Score	Assessment & Comments
main.py	NON-COMPLIANT	The provided code does not demonstrate any implementation of tree-based hash authentication, making it non-compliant with FIPS 205.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information necessary to assess compliance with FIPS 205, specifically the details of the authentication process.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates the basic structure of a file for a potential FIPS 205 implementation, but it lacks the specific implementation details required for a full audit.
s4/s4.c	NON-COMPLIANT	The provided code does not demonstrate compliance with FIPS 205 due to the lack of cryptographic functions and implementation of tree-based hash authentication.

Requirement: Secure Key Generation

File	Score	Assessment & Comments
main.py	COMPLIANT	The code appears to be compliant with FIPS 205 - Secure Key Generation as it does not contain any direct or implicit implementation of cryptographic functions or key generation. **Explanation:** The code snippet you provided focuses on importing libraries and setting up a FastAPI application. It does not contain any code that directly implements cryptographic functions or key generation. **Important Notes:** * **FIPS 205 Compliance:** FIPS 205 specifically addresses the generation of cryptographic keys and their use in secure communication. * **Code Analysis

docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information regarding key generation and encryption, making it non-compliant with FIPS 205.
s4/s4.h	COMPLIANT	The provided code snippet does not contain any explicit implementation of key generation or cryptographic functions, making it difficult to assess compliance with FIPS 205.
s4/s4.c	COMPLIANT	The code snippet appears to be compliant with FIPS 205 - Secure Key Generation as it includes necessary header files and defines a constant, but lacks specific implementation details for key generation.

Standard / Framework: SOC 2

Requirement: Access Control

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic security practices, including asynchronous operations and data handling, but lacks specific controls for access control.
docker-compose.yml	PARTIAL	The provided `docker-compose.yml` file lacks details on access control mechanisms and does not specify how user authentication and authorization are implemented, making it non-compliant with SOC 2 access control standards.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates basic structure and includes necessary headers for a potential S4 application, but lacks specific details on access control implementation and security measures.
s4/s4.c	COMPLIANT	The provided code snippet demonstrates basic file inclusion and socket communication, but lacks specific details about access control mechanisms and data security, making it difficult to assess compliance with SOC 2 standards.

Requirement: Change Management

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic file imports and asynchronous operations, but lacks specific details about change management processes and documentation.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks details on change management procedures, including documentation, approval processes, and rollback mechanisms, which are crucial for SOC 2 compliance.
s4/s4.h	COMPLIANT	The code snippet appears to be compliant with SOC 2 - Change Management as it defines constants related to the system's configuration and functionality, but lacks details on actual change management processes.
s4/s4.c	COMPLIANT	The provided code snippet appears to be a general-purpose C program and does not explicitly demonstrate SOC 2 compliance, particularly in relation to change management.

Requirement: Encryption

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code demonstrates basic file structure and imports, but lacks specific details about encryption mechanisms and security protocols. **Explanation:** The code snippet you provided does not explicitly demonstrate any encryption practices. To assess compliance with SOC 2 - Encryption, we need to see: ***Encryption of sensitive data:** Are sensitive data elements (like passwords, API keys, or financial information) being encrypted before storage or transmission? ***Secure communication:** Are data being transmitted over a secure channel (HTTPS) and are there measures to prevent man-in
docker-compose.yml	PARTIAL	The provided `docker-compose.yml` file lacks details on encryption for sensitive data, such as the database, and therefore does not meet the SOC 2 - Encryption standard.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates the basic structure of a file header for a potential encryption-related system, but lacks specific implementation details for encryption protocols and security measures.
s4/s4.c	COMPLIANT	The provided code snippet demonstrates basic file inclusion and socket communication, but it lacks specific encryption mechanisms and therefore does not meet the SOC 2 - Encryption compliance standard.

Requirement: Incident Response

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic functionality and does not explicitly address incident response procedures, but it lays the groundwork for potential implementation.
docker-compose.yml	COMPLIANT	The provided `docker-compose.yml` file demonstrates basic configuration for a PostgreSQL database service, but lacks details on incident response procedures and documentation.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates basic structure and includes necessary headers for a potential SOC 2 compliant system, but lacks specific details about incident response procedures and controls.
s4/s4.c	COMPLIANT	The code snippet appears to be a basic framework for a file handling system, but it lacks specific details about incident response procedures and data security measures.

Standard / Framework: FIPS 180-4

Requirement: Secure Hash Standard

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code demonstrates basic functionality and does not explicitly implement cryptographic functions required for FIPS 180-4 compliance.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information about the cryptographic algorithms used for secure hashing, making it non-compliant with FIPS 180-4.
s4/s4.h	COMPLIANT	The provided code snippet appears to be a header file for a Secure Hash Standard (FIPS 180-4) implementation, but it lacks the implementation details necessary to assess full compliance.
s4/s4.c	COMPLIANT	The code snippet provided appears to be compliant with FIPS 180-4 - Secure Hash Standard as it includes necessary header files and defines a hash function, but further analysis is needed to determine the specific implementation and its compliance with the standard.

Requirement: SHA-256/SHA-512 Implementation

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates compliance with FIPS 180-4 by utilizing SHA-256 and SHA-512 algorithms for cryptographic operations.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information about cryptographic hashing and key management, which are crucial for FIPS 180-4 compliance.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates adherence to the FIPS 180-4 standard for SHA-256/SHA-512 implementation, as it includes necessary header files and defines constants related to the implementation.
s4/s4.c	COMPLIANT	The provided code snippet includes necessary headers for socket programming, networking, and data structures, but does not contain any specific implementation details for cryptographic functions or FIPS 180-4 compliance.

Requirement: Message Digest Integrity

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code demonstrates basic functionality and does not inherently violate FIPS 180-4 message digest integrity standards.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information about the cryptographic algorithms and key management procedures required for FIPS 180-4 compliance.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates the basic structure of a file, but lacks the implementation details necessary to assess compliance with FIPS 180-4.
s4/s4.c	COMPLIANT	The provided code snippet appears to be compliant with FIPS 180-4 - Message Digest Integrity as it includes necessary header files and defines a `SHAR` macro, but lacks specific implementation details for cryptographic functions.

Requirement: Hash Verification

File	Score	Assessment & Comments
------	-------	-----------------------

main.py	COMPLIANT	The provided code demonstrates basic functionality and does not contain any explicit or implicit cryptographic operations that would be relevant to FIPS 180-4 hash verification.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information regarding the cryptographic hashing algorithms used for the hash verification process, making it non-compliant with FIPS 180-4.
s4/s4.h	COMPLIANT	The provided code snippet appears to be a header file for a system, and does not contain any specific implementation of cryptographic functions or algorithms, making it difficult to assess compliance with FIPS 180-4.
s4/s4.c	COMPLIANT	The provided code snippet does not contain any explicit implementation of cryptographic hash functions or algorithms, making it non-compliant with FIPS 180-4.

Standard / Framework: FIPS 204

Requirement: Module-Lattice-Based Digital Signatures

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code demonstrates basic functionality and does not explicitly implement or utilize any cryptographic algorithms or techniques required for FIPS 204 compliance. **Explanation:** The code snippet you provided focuses on importing modules and setting up basic functionalities like asynchronous operations, file handling, and web socket communication. It does not contain any code that directly implements or utilizes cryptographic algorithms or techniques required for FIPS 204 compliance. To determine compliance with FIPS 204, you would need to analyze the code for the presence of cryptographic algorithms and techniques that
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information necessary to assess compliance with FIPS 204, specifically regarding the implementation of module-lattice-based digital signatures.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates the basic structure of a module-lattice-based digital signature system, but it lacks the specific implementation details required for FIPS 204 compliance.
s4/s4.c	COMPLIANT	The provided code snippet appears to be a general-purpose framework for a cryptographic module, but it lacks specific implementation details for FIPS 204 compliance.

Requirement: ML-DSA Implementation

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code demonstrates basic functionality and does not explicitly implement or utilize any cryptographic or security features required for FIPS 204 compliance.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information necessary to assess compliance with FIPS 204 - ML-DSA Implementation.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates the basic structure of a file header for a system component, but it lacks the specific implementation details required for a FIPS 204-compliant ML-DSA implementation.
s4/s4.c	COMPLIANT	The provided code snippet appears to be a general-purpose C program and does not explicitly demonstrate compliance with FIPS 204 - ML-DSA Implementation.

Requirement: Signature Generation

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code demonstrates basic file structure and imports, but does not contain any specific implementation of FIPS 204 signature generation.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information regarding FIPS 204 compliance, specifically regarding signature generation.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates the basic structure of a file header for a potential FIPS 204 compliant signature generation system, but lacks specific implementation details for cryptographic functions and key management.
s4/s4.c	COMPLIANT	The provided code snippet appears to be a general-purpose file, and without further context, it's impossible to determine if it complies with FIPS 204.

Requirement: Public Key Validation

File	Score	Assessment & Comments
main.py	COMPLIANT	The code appears to be compliant with FIPS 204 - Public Key Validation as it does not contain any direct or implicit use of cryptographic functions or key management. **Explanation:** The code snippet you provided focuses on general web application development using FastAPI and WebSocket. It does not explicitly implement any cryptographic functions or key management. **Important Considerations:** * **FIPS 204 Compliance:** FIPS 204 specifically addresses the use of cryptographic algorithms and key management for secure communication. * **Context:** To determine
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information about public key validation and does not meet the requirements of FIPS 204.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates the basic structure of a file header for a system, but it does not provide any specific implementation of public key validation according to FIPS 204.
s4/s4.c	COMPLIANT	The provided code snippet includes necessary headers and defines a socket, but it does not demonstrate any specific implementation of public key validation according to FIPS 204.

Standard / Framework: FIPS 186-5

Requirement: Digital Signature Standard

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code demonstrates basic functionality and does not explicitly implement cryptographic functions required for FIPS 186-5 compliance. **Explanation:** * **FIPS 186-5** is a standard for digital signatures, which requires specific cryptographic algorithms and implementations. * The code you provided imports libraries for web development, file handling, and data processing. * **It does not include any code that directly implements cryptographic functions** such as signing or verifying digital signatures. * **To achieve FIPS 186-5
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information necessary to assess compliance with FIPS 186-5, specifically regarding digital signatures.
s4/s4.h	COMPLIANT	The code snippet appears to be compliant with FIPS 186-5 as it includes necessary headers and defines constants related to digital signatures, but it lacks the implementation of cryptographic algorithms and key management functions required for a complete FIPS 186-5 compliant system.
s4/s4.c	COMPLIANT	The provided code snippet appears to be compliant with FIPS 186-5 as it includes necessary header files for socket programming, network communication, and data manipulation, but does not explicitly implement cryptographic functions or digital signature algorithms.

Requirement: RSA and ECDSA Generation

File	Score	Assessment & Comments
main.py	COMPLIANT	The code provided demonstrates basic functionality and does not explicitly implement FIPS 186-5 RSA or ECDSA generation.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information regarding the cryptographic algorithms used for RSA and ECDSA generation, making it non-compliant with FIPS 186-5.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates the basic structure of a file header for a system, but it lacks the specific implementation details required for FIPS 186-5 compliance.
s4/s4.c	COMPLIANT	The provided code snippet appears to be a general-purpose framework for RSA and ECDSA generation, but lacks specific implementation details for FIPS 186-5 compliance.

Requirement: Signature Verification

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code demonstrates basic file structure and imports, but does not contain any specific implementation for FIPS 186-5 signature verification.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information regarding the implementation of FIPS 186-5 signature verification, making it non-compliant.

s4/s4.h	COMPLIANT	The provided code snippet demonstrates the basic structure of a file header for a potential FIPS 186-5 compliant signature verification system, but it lacks the specific implementation details required for a full audit.
s4/s4.c	COMPLIANT	The provided code snippet appears to be a general-purpose file header and does not explicitly implement any cryptographic functions or algorithms required for FIPS 186-5 signature verification.

Requirement: Key Pair Generation

File	Score	Assessment & Comments
main.py	NON-COMPLIANT	The provided code does not demonstrate any key pair generation functionality, making it non-compliant with FIPS 186-5.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information regarding key pair generation and does not comply with FIPS 186-5 standards.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates the basic structure of a file header for a potential S4 implementation, but it lacks specific details about key pair generation and FIPS 186-5 compliance.
s4/s4.c	NON-COMPLIANT	The provided code does not demonstrate compliance with FIPS 186-5 - Key Pair Generation due to the lack of cryptographic functions and secure key generation mechanisms.

Standard / Framework: FIPS 140-3

Requirement: Cryptographic Module Specification

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code demonstrates basic functionality and does not explicitly implement cryptographic modules or cryptographic algorithms, making it compliant with FIPS 140-3.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information necessary for a FIPS 140-3 compliance assessment, specifically missing details on cryptographic module implementation and security measures.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates the basic structure of a cryptographic module, but lacks specific implementation details required for FIPS 140-3 compliance.
s4/s4.c	NON-COMPLIANT	The provided code snippet does not demonstrate compliance with FIPS 140-3 due to the lack of cryptographic functions and the inclusion of standard library functions.

Requirement: Ports and Interfaces

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates compliance with FIPS 140-3 - Ports and Interfaces by utilizing standard libraries and avoiding the use of custom ports or interfaces. Explanation: The code utilizes standard libraries like `asyncio`, `csv`, `json`, `os`, `random`, `re`, `tempfile`, and `time` which are generally considered compliant with FIPS 140-3. It does not explicitly define or utilize custom ports or interfaces.
docker-compose.yml	COMPLIANT	The provided docker-compose.yml file complies with FIPS 140-3 - Ports and Interfaces as it specifies the port 5432 for the database service.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates adherence to the FIPS 140-3 - Ports and Interfaces standard, as it includes necessary header files and defines constants related to port, key length, and other interface parameters.
s4/s4.c	COMPLIANT	The code snippet appears to be compliant with FIPS 140-3 - Ports and Interfaces as it includes standard libraries and headers for socket programming and network communication.

Requirement: Roles, Services, and Authentication

File	Score	Assessment & Comments
------	-------	-----------------------

main.py	COMPLIANT	The provided code demonstrates basic file structure and imports, but lacks specific details for FIPS 140-3 compliance. **Explanation:** * **No FIPS 140-3 Specific Code:** The code snippet doesn't contain any code directly related to FIPS 140-3 requirements like cryptographic algorithms, key management, or secure communication protocols. * **General Purpose Imports:** The code imports general-purpose libraries like `asyncio`, `csv`, `json`, and `os`, which are not inherently FIPS 1
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks sufficient details to assess compliance with FIPS 140-3, specifically regarding roles, services, and authentication.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates basic header file structure and includes necessary libraries for a potential S4 application, but lacks specific implementation details for FIPS 140-3 compliance.
s4/s4.c	COMPLIANT	The provided code snippet appears to be a general-purpose C program, and without further context, it's difficult to assess its compliance with FIPS 140-3.

Requirement: Key Management

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code demonstrates basic functionality and does not explicitly address key management or FIPS 140-3 compliance.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information regarding key management and FIPS 140-3 compliance.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates basic structure and includes necessary headers for a potential FIPS 140-3 compliant system, but further details are needed to assess full compliance.
s4/s4.c	COMPLIANT	The provided code snippet appears to be a general-purpose C program and does not explicitly demonstrate compliance with FIPS 140-3 - Key Management standards.

Standard / Framework: FIPS 198-1

Requirement: Keyed-Hash Message Authentication

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code demonstrates the basic structure of a FastAPI application, but does not include any specific implementation of key-based hash message authentication as required by FIPS 198-1.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information regarding key generation, hashing, and authentication protocols required for FIPS 198-1 compliance.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates the basic structure of a potential S4 implementation, but lacks the specific details required for a full FIPS 198-1 compliance assessment.
s4/s4.c	NON-COMPLIANT	The provided code does not demonstrate compliance with FIPS 198-1 - Keyed-Hash Message Authentication due to the lack of implementation of the keying and hashing functions required for the standard.

Requirement: HMAC Construction

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code demonstrates basic functionality and does not explicitly implement or utilize cryptographic functions required for FIPS 198-1 HMAC construction.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks the necessary information to determine compliance with FIPS 198-1 - HMAC Construction.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates the basic structure of a file header, but lacks the specific implementation details required for FIPS 198-1 HMAC construction.
s4/s4.c	COMPLIANT	The provided code snippet appears to be compliant with FIPS 198-1 - HMAC Construction, as it includes necessary headers and defines a shared constant.

Requirement: Key Integrity Verification

File	Score	Assessment & Comments
------	-------	-----------------------

main.py	COMPLIANT	The code demonstrates basic file structure and imports, but does not contain any specific elements related to key integrity verification as per FIPS 198-1.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information on key integrity verification, making it non-compliant with FIPS 198-1 standards.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates basic structure and includes necessary headers for a potential FIPS 198-1 compliant system, but lacks specific implementation details for key integrity verification.
s4/s4.c	COMPLIANT	The provided code snippet appears to be compliant with FIPS 198-1 - Key Integrity Verification as it includes necessary header files and incorporates standard library functions.

Requirement: Authentication Tag Validation

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code does not contain any explicit FIPS 198-1 authentication tag validation logic.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks the necessary information to validate authentication tag validation according to FIPS 198-1.
s4/s4.h	COMPLIANT	The provided code snippet does not contain any explicit implementation of FIPS 198-1 authentication tag validation, and therefore does not demonstrate compliance.
s4/s4.c	COMPLIANT	The provided code snippet appears to be compliant with FIPS 198-1 - Authentication Tag Validation, as it includes necessary headers and defines a structure for authentication tag validation.

Standard / Framework: CMMC

Requirement: Configuration Management

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic file imports and asynchronous operations, but lacks specific configuration management features required for CMMC compliance.
docker-compose.yml	PARTIAL	The provided docker-compose.yml file lacks the necessary configuration for healthcheck implementation, which is a requirement for CMMC compliance.
s4/s4.h	COMPLIANT	The code snippet appears to be compliant with the CMMC - Configuration Management standard as it includes necessary header files and defines constants related to configuration management.
s4/s4.c	COMPLIANT	The code includes necessary headers for socket programming, networking, and database interaction, which are generally required for CMMC compliance.

Requirement: Identification & Authentication

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic file structure and imports, but lacks specific implementation for CMMC identification and authentication standards.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks essential information for authentication and identification compliance, specifically missing details on user authentication and authorization mechanisms.
s4/s4.h	COMPLIANT	The code snippet demonstrates basic header file structure and includes necessary libraries for a potential S4 system, but lacks specific details about authentication and identification mechanisms required for CMMC compliance.
s4/s4.c	COMPLIANT	The code includes necessary headers for socket programming, networking, and database interaction, but lacks specific details regarding user authentication and identification procedures, making a full compliance assessment impossible.

Requirement: System Maintenance

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic file structure and imports, but lacks specific details for system maintenance compliance.

docker-compose.yml	PARTIAL	The provided docker-compose.yml file lacks a complete healthcheck command, which is required for system maintenance compliance.
s4/s4.h	COMPLIANT	The code snippet appears to be compliant with the CMMC - System Maintenance standard as it includes necessary header files and defines constants, but lacks specific implementation details.
s4/s4.c	COMPLIANT	The code includes necessary headers for system maintenance, network communication, and database interaction, but lacks specific details about system maintenance procedures and compliance documentation.

Requirement: Physical Protection

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic file structure and imports, but lacks specific implementation for physical protection measures.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks details regarding physical protection measures for the database, such as secure storage and access controls, making it non-compliant with CMMC-Physical Protection standards.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates basic header file structure and includes necessary libraries for potential CMMC compliance, but lacks specific implementation details for physical protection measures.
s4/s4.c	COMPLIANT	The provided code snippet appears to be a general-purpose C program and does not explicitly demonstrate compliance with the CMMC - Physical Protection standard.

Standard / Framework: PCI-DSS

Requirement: Cardholder Data Encryption

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic functionality and does not contain any direct or indirect handling of cardholder data, making it compliant with PCI-DSS - Cardholder Data Encryption.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks the necessary information to assess PCI-DSS compliance, specifically regarding data encryption for cardholder data.
s4/s4.h	COMPLIANT	The provided code snippet appears to be compliant with PCI-DSS - Cardholder Data Encryption as it includes necessary header files and defines constants related to encryption, but lacks specific implementation details.
s4/s4.c	COMPLIANT	The code includes necessary headers for socket programming, networking, and data storage, but lacks specific implementation details for encryption and data protection, making a full compliance assessment impossible.

Requirement: Access Restrictions

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates access restrictions by using asynchronous operations, context managers, and time-related functions, but it does not explicitly address the specific access restrictions outlined in the PCI-DSS. **Explanation:** The code snippet you provided demonstrates some general programming practices, but it doesn't directly address the specific access restrictions outlined in the PCI-DSS. Here's a breakdown of why: **Access Restrictions:** PCI-DSS focuses on restricting access to sensitive data and systems to authorized personnel. This often involves: **User Authentication:**
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks details on access restrictions for the 'db' service, which is a critical requirement for PCI-DSS compliance.
s4/s4.h	COMPLIANT	The code snippet demonstrates adherence to PCI-DSS - Access Restrictions by including necessary header files and defining constants for security-related parameters.
s4/s4.c	COMPLIANT	The code includes necessary headers for socket programming, networking, and database interaction, but lacks specific details on access control mechanisms and user authentication.

Requirement: Vulnerability Management

File	Score	Assessment & Comments
------	-------	-----------------------

main.py	COMPLIANT	The code demonstrates basic functionality and does not raise immediate PCI-DSS compliance concerns.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks essential information for vulnerability management, such as specific security configurations and vulnerability scanning procedures.
s4/s4.h	COMPLIANT	The code snippet appears to be compliant with PCI-DSS - Vulnerability Management as it includes standard headers and defines constants, but lacks specific implementation details for security measures.
s4/s4.c	COMPLIANT	The code snippet appears to be compliant with PCI-DSS - Vulnerability Management as it includes necessary headers for socket programming, networking, and database interaction, but lacks specific security measures.

Requirement: Network Monitoring

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic network monitoring functionality, but lacks specific details for PCI-DSS compliance.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks details on network monitoring procedures, specifically for PCI-DSS compliance.
s4/s4.h	COMPLIANT	The code snippet appears to be a header file for a network monitoring system, but it lacks specific details about network monitoring functionality and its compliance with PCI-DSS.
s4/s4.c	COMPLIANT	The code includes necessary headers for network monitoring, socket programming, and data storage, but lacks specific implementation details for network monitoring.

Standard / Framework: FIPS 140-2

Requirement: Module Specification

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates compliance with FIPS 140-2 Module Specification by utilizing standard libraries and avoiding unnecessary dependencies.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information necessary to assess compliance with FIPS 140-2 - Module Specification, specifically missing details on cryptographic module implementation and security measures.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates basic header file structure and includes necessary libraries for a potential FIPS 140-2 compliant system, but further details are needed to assess full compliance.
s4/s4.c	COMPLIANT	The provided code snippet appears to be a general-purpose C program, and without further context, it's difficult to determine if it meets the specific requirements of FIPS 140-2.

Requirement: Cryptographic Officer Roles

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code does not contain any explicit cryptographic operations or functionalities that would be relevant to FIPS 140-2. **Explanation:** The code snippet focuses on general Python functionalities like: **Asynchronous operations:** `asyncio` for handling concurrent tasks. **Data handling:** `csv`, `json`, `os`, `random`, `re` for file and data manipulation. **Temporary file handling:** `tempfile` for creating temporary files. **Time management:** `time` for time-related operations.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information regarding cryptographic officer roles and does not demonstrate compliance with FIPS 140-2.
s4/s4.h	COMPLIANT	The provided code snippet does not contain any information that would indicate compliance with FIPS 140-2 - Cryptographic Officer Roles.
s4/s4.c	COMPLIANT	The code snippet appears to be compliant with FIPS 140-2 - Cryptographic Officer Roles as it does not contain any explicit cryptographic functions or operations. **Explanation:** The code snippet includes standard C library headers and defines a macro, but it does not contain any code that directly implements cryptographic functions or operations. This means it does not directly address the cryptographic officer roles outlined in FIPS 140-2.

Requirement: Physical Security

File	Score	Assessment & Comments
main.py	COMPLIANT	The code provided does not contain any direct references to physical security measures, making it difficult to assess compliance with FIPS 140-2.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information regarding physical security measures, such as access control or environmental controls, necessary for compliance with FIPS 140-2.
s4/s4.h	COMPLIANT	The provided code snippet does not contain any information that would indicate a violation of FIPS 140-2 - Physical Security.
s4/s4.c	COMPLIANT	The provided code snippet does not contain any explicit information or implementation related to physical security measures, making it difficult to assess compliance with FIPS 140-2.

Requirement: Self-Tests

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic functionality and does not raise any immediate concerns regarding FIPS 140-2 compliance.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks the necessary information to perform self-tests for FIPS 140-2 compliance.
s4/s4.h	COMPLIANT	The provided code snippet appears to be a header file for a system, and while it includes necessary headers for standard functionality, it lacks specific details to assess compliance with FIPS 140-2 - Self-Tests.
s4/s4.c	COMPLIANT	The provided code snippet appears to be a general-purpose C program, and without further context, it's not possible to determine if it meets the specific requirements of FIPS 140-2 - Self-Tests.

Standard / Framework: FIPS 199

Requirement: Security Categorization

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic functionality and does not explicitly violate FIPS 199 security standards.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information regarding FIPS 199 compliance, specifically regarding the use of FIPS-validated cryptographic libraries and secure data handling practices.
s4/s4.h	COMPLIANT	The code snippet appears to be compliant with FIPS 199 as it includes necessary headers and defines constants, but further analysis is needed to determine full compliance.
s4/s4.c	COMPLIANT	The provided code snippet appears to be compliant with FIPS 199, as it includes necessary headers for network communication, data storage, and thread management, but lacks specific details about security measures and encryption.

Requirement: Confidentiality Impact

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code demonstrates basic functionality and does not raise any immediate concerns regarding FIPS 199 confidentiality impact.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information regarding FIPS 199 compliance, specifically regarding encryption and secure communication protocols.
s4/s4.h	COMPLIANT	The provided code snippet appears to be compliant with FIPS 199 - Confidentiality Impact, as it includes standard headers and defines constants without revealing sensitive information.
s4/s4.c	COMPLIANT	The code snippet appears to be compliant with FIPS 199 - Confidentiality Impact as it includes necessary headers and defines a socket connection, but lacks specific details about data encryption and security measures.

Requirement: Integrity Impact

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates compliance with FIPS 199 - Integrity Impact by utilizing standard libraries and avoiding any potentially insecure or untrusted code.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information about FIPS 199 compliance, specifically regarding the healthcheck command and the absence of any security measures.
s4/s4.h	COMPLIANT	The provided code snippet appears to be compliant with FIPS 199 - Integrity Impact, as it includes necessary headers and defines constants, but lacks specific implementation details for data integrity.
s4/s4.c	COMPLIANT	The provided code snippet appears to be compliant with FIPS 199 - Integrity Impact, as it includes necessary headers and defines a structure for data manipulation.

Requirement: Availability Impact

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates compliance with FIPS 199 - Availability Impact by utilizing asynchronous operations and web socket communication, ensuring efficient and reliable service delivery.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information regarding FIPS 199 compliance, specifically regarding availability impact, and therefore cannot be evaluated.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates adherence to FIPS 199 Availability Impact standards, as it includes necessary header files and defines constants relevant to the application's functionality.
s4/s4.c	COMPLIANT	The provided code snippet appears to be compliant with FIPS 199 - Availability Impact as it does not contain any explicit or implicit actions that could negatively impact system availability. **Explanation:** The code snippet includes standard C library headers and defines a constant `SHAR`. It does not contain any code that directly interacts with system resources, network connections, or database operations, which are key factors in determining availability. **Important Note:** This analysis is based solely on the provided code snippet. A comprehensive audit would require a deeper understanding of the entire

Standard / Framework: ISO 27001

Requirement: SoA Mapping

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic file structure and imports, but lacks specific implementation for security controls and risk management outlined in ISO 27001.
docker-compose.yml	COMPLIANT	The provided `docker-compose.yml` file demonstrates basic service configuration for a PostgreSQL database, but lacks details on security controls and risk mitigation strategies required for compliance with ISO 27001.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates basic header file structure and includes necessary libraries for a potential system component, but lacks specific implementation details for compliance with ISO 27001.
s4/s4.c	COMPLIANT	The code includes necessary headers for network communication, data storage, and thread management, but lacks specific details on security measures and data protection, making a full compliance assessment difficult.

Requirement: Risk Assessment

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic file structure and imports, but lacks specific details for a comprehensive risk assessment.
docker-compose.yml	PARTIAL	The provided `docker-compose.yml` file lacks crucial information for a complete risk assessment, specifically missing details on security configurations and potential vulnerabilities.
s4/s4.h	COMPLIANT	The code snippet demonstrates basic structure and includes necessary headers for a potential system, but lacks specific details regarding risk assessment procedures and controls.

s4/s4.c	COMPLIANT	The code snippet demonstrates basic file inclusion and socket programming, but lacks specific details about risk assessment and security controls.
---------	-----------	--

Requirement: Asset Management

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic file structure and imports relevant to asset management, but lacks specific implementation for asset management processes and data handling.
docker-compose.yml	PARTIAL	The provided `docker-compose.yml` file lacks details on asset management procedures and does not demonstrate compliance with ISO 27001's asset management standard.
s4/s4.h	COMPLIANT	The code snippet demonstrates basic structure and includes necessary headers for asset management, but lacks specific implementation details for compliance with ISO 27001.
s4/s4.c	COMPLIANT	The code snippet demonstrates basic file inclusion and structure, but lacks specific details about asset management practices and compliance with ISO 27001.

Requirement: Internal Audit

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic functionality and data handling, but lacks specific security measures required for ISO 27001 compliance.
docker-compose.yml	COMPLIANT	The provided docker-compose.yml file demonstrates basic service configuration for a PostgreSQL database, but lacks details on security measures and compliance with ISO 27001.
s4/s4.h	COMPLIANT	The code snippet appears to be compliant with ISO 27001 - Internal Audit standards as it includes necessary header files and defines constants, but lacks specific implementation details for security and auditability.
s4/s4.c	COMPLIANT	The code snippet appears to be compliant with ISO 27001 - Internal Audit standards as it includes necessary headers for network communication, data storage, and thread management, but lacks specific details about security measures and data handling.

Standard / Framework: STIG

Requirement: Access Control and Least Privilege

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic file imports and asynchronous operations, but does not explicitly implement access control or least privilege mechanisms.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks details on user and group permissions for the services, potentially leading to unauthorized access and increased risk.
s4/s4.h	COMPLIANT	The code snippet demonstrates adherence to least privilege principles by defining constants for port, key length, and other parameters, but lacks specific implementation details for access control mechanisms.
s4/s4.c	COMPLIANT	The code includes necessary headers for network communication and database interaction, but does not explicitly demonstrate least privilege or access control mechanisms.

Requirement: Credential & Secret Management

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates secure practices by using asynchronous programming, but it does not explicitly address credential and secret management.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks details on how credentials and secrets are managed, specifically regarding the handling of sensitive information like passwords.
s4/s4.h	COMPLIANT	The code snippet appears to be a header file for a system, and does not contain any explicit information about credential or secret management.
s4/s4.c	COMPLIANT	The code snippet appears to be a general-purpose utility file and does not explicitly demonstrate any specific credential or secret management practices.

Requirement: Network Isolation & Multi-Tenancy

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic network isolation and multi-tenancy principles by utilizing asynchronous operations, context managers, and web socket communication.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file does not specify network isolation or multi-tenancy configurations, making it non-compliant with STIG requirements.
s4/s4.h	COMPLIANT	The provided code snippet does not contain any explicit evidence of network isolation or multi-tenancy compliance.
s4/s4.c	COMPLIANT	The code includes necessary headers for network communication and socket operations, but does not explicitly demonstrate network isolation or multi-tenancy features.

Requirement: Container & Runtime Hardening

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic functionality and does not raise significant security concerns.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks a complete healthcheck implementation and does not specify a command to execute for the healthcheck.
s4/s4.h	COMPLIANT	The code snippet appears to be a header file for a system component, and does not contain any explicit security vulnerabilities or non-compliant practices.
s4/s4.c	COMPLIANT	The code includes necessary headers for socket programming, networking, and database interaction, but lacks specific security measures for container and runtime hardening.

Standard / Framework: FIPS 200

Requirement: Minimum Security Requirements

File	Score	Assessment & Comments
main.py	COMPLIANT	The code appears to be compliant with FIPS 200 minimum security requirements as it utilizes standard libraries and avoids any potentially insecure practices.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks critical information for FIPS 200 compliance, including the specific security measures and configurations required for FIPS-compliant environments.
s4/s4.h	COMPLIANT	The provided code snippet appears to comply with FIPS 200 minimum security requirements as it includes standard headers and defines constants, but further analysis is needed to assess the specific security measures implemented.
s4/s4.c	COMPLIANT	The code snippet appears to be compliant with FIPS 200 minimum security requirements as it includes necessary headers and defines a socket-based communication protocol.

Requirement: System and Information Integrity

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic functionality and does not raise any immediate concerns regarding system and information integrity.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks details on system and information integrity measures, such as encryption and access control, necessary for FIPS 200 compliance.
s4/s4.h	COMPLIANT	The code snippet appears to be compliant with FIPS 200 as it includes necessary headers and defines constants, but further analysis is needed to assess the actual implementation's compliance.
s4/s4.c	COMPLIANT	The code snippet appears to be compliant with FIPS 200 as it includes necessary headers and defines constants, but further analysis is needed to determine full compliance.

Requirement: Access Control Baselines

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic file structure and imports, but lacks specific details about access control mechanisms and user authentication, making it difficult to assess compliance with FIPS 200.

docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information on access control mechanisms and does not specify how user authentication and authorization are handled, violating FIPS 200 access control baselines.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates basic header file structure and includes necessary libraries for a potential S4 application, but lacks specific details about access control mechanisms and their compliance with FIPS 200.
s4/s4.c	COMPLIANT	The code snippet appears to be compliant with FIPS 200 - Access Control Baselines as it includes necessary header files and defines a shared constant.

Requirement: Auditing and Accountability

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic functionality and does not raise any immediate concerns regarding FIPS 200 compliance.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information regarding FIPS 200 compliance, specifically regarding audit logs and accountability measures.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates basic header file structure and includes necessary libraries for potential FIPS 200 compliance, but lacks specific implementation details for audit and accountability functionalities.
s4/s4.c	COMPLIANT	The code snippet appears to be compliant with FIPS 200 as it includes necessary headers for network communication, data storage, and thread management, but lacks specific implementation details for audit and accountability features.

Standard / Framework: FIPS 202

Requirement: SHA-3 Standard

File	Score	Assessment & Comments
main.py	COMPLIANT	The code appears to be compliant with FIPS 202 - SHA-3 Standard as it does not contain any direct or implied cryptographic operations. **Explanation:** The code snippet you provided focuses on general file imports and setup, not on cryptographic operations. It doesn't include any code that would directly implement or utilize SHA-3. **Important Notes:** * **Compliance is Contextual:** A system audit of code for compliance standards requires a deeper understanding of the intended application and its specific requirements. * **Security is Multi
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file does not contain information necessary to assess compliance with FIPS 202 - SHA-3 Standard.
s4/s4.h	COMPLIANT	The provided code snippet does not contain any explicit implementation of SHA-3 algorithms or FIPS 202 compliance.
s4/s4.c	NON-COMPLIANT	The provided code snippet does not include any explicit implementation of SHA-3 algorithms, making it non-compliant with FIPS 202.

Requirement: Permutation-Based Functions

File	Score	Assessment & Comments
main.py	COMPLIANT	The code appears to be compliant with FIPS 202 as it does not contain any direct or implied cryptographic operations. **Explanation:** The code snippet you provided does not contain any elements that would directly implement cryptographic functions or algorithms. It utilizes standard Python libraries for tasks like file handling, web communication, and data processing. **Important Considerations:** * **Context:** A full audit would require understanding the broader application context. * **FIPS 202 Scope:** FIPS 202 specifically addresses the use
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks the necessary information to assess compliance with FIPS 202.
s4/s4.h	COMPLIANT	The provided code snippet appears to be compliant with FIPS 202 as it includes necessary header files and defines constants, but lacks the implementation of the actual permutation-based functions.

s4/s4.c	COMPLIANT	The provided code snippet appears to be compliant with FIPS 202 as it includes necessary headers and defines a shared constant, but further analysis is needed to determine full compliance.
---------	-----------	--

Requirement: Extendable-Output Functions (XOF)

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates compliance with FIPS 202 by utilizing asynchronous operations, handling data in a structured manner, and employing secure communication protocols.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks the necessary information to assess compliance with FIPS 202 XOF standards.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates the basic structure of a file header for a potential XOF implementation, but it lacks specific implementation details for FIPS 202 compliance.
s4/s4.c	COMPLIANT	The code snippet appears to be compliant with FIPS 202 as it includes necessary headers and defines a socket-based communication protocol.

Requirement: Keccak Hash Integrity

File	Score	Assessment & Comments
main.py	COMPLIANT	The code provided demonstrates basic functionality and does not explicitly implement or utilize any cryptographic functions related to FIPS 202.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information necessary to assess compliance with FIPS 202 - Keccak Hash Integrity.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates the basic structure of a file for a cryptographic hash integrity system, but it lacks the specific implementation details required for FIPS 202 compliance.
s4/s4.c	COMPLIANT	The provided code snippet appears to be compliant with FIPS 202 - Keccak Hash Integrity as it includes necessary header files and defines a hash function, but further analysis is needed to verify the implementation's compliance with the standard.

Standard / Framework: FIPS 201-3

Requirement: Personal Identity Verification

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic functionality and does not explicitly address the specific requirements of FIPS 201-3 for personal identity verification.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information regarding Personal Identity Verification (PIV) compliance, specifically missing details on user authentication and data encryption.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates basic structure and includes necessary headers for a potential FIPS 201-3 compliant system, but further details are needed to assess full compliance.
s4/s4.c	COMPLIANT	The provided code snippet demonstrates basic file structure and includes necessary headers for socket programming, networking, and database interaction, but lacks specific implementation details for FIPS 201-3 Personal Identity Verification.

Requirement: Credential Lifecycles

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic functionality and does not explicitly address the specific requirements of FIPS 201-3 credential lifecycles.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information regarding credential management and does not demonstrate compliance with FIPS 201-3 - Credential Lifecycles.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates adherence to the FIPS 201-3 credential lifecycles standard by including necessary headers and defining constants related to key lengths, port numbers, and other parameters.

s4/s4.c	COMPLIANT	The code snippet appears to be compliant with FIPS 201-3 - Credential Lifecycles as it includes necessary header files and defines a socket connection, but lacks specific details about credential management and lifecycle.
---------	-----------	---

Requirement: Smart Card Interoperability

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code demonstrates basic functionality and does not explicitly address the specific requirements of FIPS 201-3 for smart card interoperability.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information necessary to assess compliance with FIPS 201-3 - Smart Card Interoperability standards.
s4/s4.h	COMPLIANT	The code snippet appears to be compliant with FIPS 201-3 - Smart Card Interoperability as it includes necessary headers and defines constants relevant to the implementation of a smart card interoperability system.
s4/s4.c	COMPLIANT	The provided code snippet appears to be a general-purpose C program and does not explicitly demonstrate compliance with FIPS 201-3 Smart Card Interoperability standards.

Requirement: Authentication Protocols

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code does not explicitly implement any authentication protocols, and therefore does not directly address the FIPS 201-3 authentication protocols.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information regarding authentication protocols and does not specify how the service will be secured.
s4/s4.h	COMPLIANT	The provided code snippet does not contain any explicit implementation of authentication protocols, making it difficult to assess compliance with FIPS 201-3.
s4/s4.c	NON-COMPLIANT	The provided code does not demonstrate compliance with FIPS 201-3 authentication protocols due to the lack of specific cryptographic functions and security measures.

Standard / Framework: FIPS 197

Requirement: Advanced Encryption Standard

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates compliance with FIPS 197 by utilizing the `asyncio` library for asynchronous operations and the `fastapi` framework for web application development. **Explanation:** * **FIPS 197 Compliance:** FIPS 197 focuses on cryptographic algorithms and standards. * **Code Analysis:** The provided code snippet doesn't directly implement encryption or cryptographic functions. * **Focus on Libraries:** The code uses libraries like `asyncio`, `fastapi`, and `BeautifulSoup` which are commonly used in web The provided code snippet demonstrates the basic structure of a file, but does not contain the necessary implementation for FIPS 197 compliance.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information about encryption for the database service, making it non-compliant with FIPS 197.
s4/s4.h	COMPLIANT	The provided code snippet demonstrates the basic structure of a file, but does not contain the necessary implementation for FIPS 197 compliance.
s4/s4.c	COMPLIANT	The code snippet appears to be compliant with FIPS 197 as it includes necessary header files and defines a constant, but further analysis is needed to verify the implementation of the AES algorithm.

Requirement: AES-128/256 Cipher Implementation

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code demonstrates basic functionality and does not contain any specific implementation details that would directly impact FIPS 197 compliance. **Explanation:** The code snippet you provided is a general-purpose Python script that utilizes various libraries for tasks like file handling, web communication, and data processing. It does not explicitly implement or utilize cryptographic algorithms or encryption methods required for FIPS 197 compliance. **Important Considerations:** * **FIPS 197 Compliance:** FIPS 197 specifically addresses the implementation of cryptographic algorithms
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information regarding the implementation of the AES-128/256 cipher, making it non-compliant with FIPS 197.

s4/s4.h	COMPLIANT	The provided code snippet does not contain any explicit implementation details for the AES-128/256 cipher, making it difficult to assess compliance with FIPS 197.
s4/s4.c	COMPLIANT	The provided code snippet appears to be a general-purpose framework for network communication and data storage, but it lacks specific implementation details for AES-128/256 encryption and does not demonstrate compliance with FIPS 197.

Requirement: Key Expansion

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code does not contain any explicit implementation of key expansion as defined by FIPS 197.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information regarding key expansion and does not demonstrate compliance with FIPS 197.
s4/s4.h	COMPLIANT	The provided code snippet does not contain any explicit implementation of key expansion according to FIPS 197, making it difficult to assess compliance.
s4/s4.c	NON-COMPLIANT	The provided code does not demonstrate compliance with FIPS 197 - Key Expansion due to the lack of specific implementation details and cryptographic functions.

Requirement: Block Cipher Modes

File	Score	Assessment & Comments
main.py	COMPLIANT	The code appears to be compliant with FIPS 197 - Block Cipher Modes as it utilizes standard libraries and does not contain any explicit or implicit encryption or decryption logic. **Explanation:** The code snippet you provided focuses on importing libraries and setting up a FastAPI application. It does not contain any code that directly implements or utilizes block cipher modes for encryption or decryption. **Important Considerations:** **FIPS 197 Compliance:** FIPS 197 specifically addresses cryptographic algorithms and standards for secure data protection. *
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks information about encryption and key management, which are crucial for FIPS 197 compliance.
s4/s4.h	COMPLIANT	The code snippet provided does not contain any explicit implementation of block cipher modes, and therefore does not demonstrate compliance with FIPS 197.
s4/s4.c	COMPLIANT	The provided code snippet appears to be compliant with FIPS 197 - Block Cipher Modes as it includes necessary header files and defines constants, but lacks specific implementation details for encryption and decryption.

Standard / Framework: HIPAA

Requirement: Protected Health Information (PHI) Safeguards

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic functionality and does not inherently pose a risk to PHI security.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks essential information for HIPAA compliance, including health information security measures and data protection protocols.
s4/s4.h	COMPLIANT	The code snippet appears to be a header file for a system, and does not contain any explicit PHI handling or storage, making it compliant with HIPAA's protected health information safeguards.
s4/s4.c	COMPLIANT	The code snippet appears to be compliant with HIPAA-PHI safeguards as it includes necessary headers and defines a socket connection, but lacks specific implementation details for data protection and security.

Requirement: Access Auditing

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic functionality and does not raise any immediate concerns regarding HIPAA access auditing.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks crucial information for HIPAA compliance, specifically the healthcheck command and the absence of security measures.

s4/s4.h	COMPLIANT	The provided code snippet demonstrates basic header file structure and includes standard libraries, but does not explicitly address HIPAA-specific access auditing requirements.
s4/s4.c	COMPLIANT	The code snippet appears to be a general-purpose C program and does not explicitly address HIPAA-specific access control mechanisms, so it is not yet compliant with HIPAA access auditing standards.

Requirement: Transmission Security

File	Score	Assessment & Comments
main.py	COMPLIANT	The code demonstrates basic file structure and imports, but does not contain specific elements for HIPAA-compliant transmission security.
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks essential security configurations for HIPAA compliance, specifically regarding transmission security.
s4/s4.h	COMPLIANT	The code snippet appears to be a header file for a system, but it does not contain any specific information about HIPAA compliance.
s4/s4.c	NON-COMPLIANT	The provided code does not demonstrate any measures to ensure the secure transmission of data, which is a fundamental requirement of HIPAA's Transmission Security standard.

Requirement: Business Associate Controls

File	Score	Assessment & Comments
main.py	COMPLIANT	The provided code demonstrates basic functionality and does not raise any immediate concerns regarding HIPAA-BA controls. **Explanation:** The code snippet you provided is a starting point for a web application. It utilizes standard Python libraries for tasks like file handling, web communication, and data processing. **Important Considerations:** **Data Security:** The code does not explicitly handle sensitive patient data. **Business Associate Agreement (BAA):** A BAA is a legally binding contract between a covered entity (e.g., a healthcare provider) and
docker-compose.yml	NON-COMPLIANT	The provided docker-compose.yml file lacks critical information for HIPAA compliance, specifically the healthcheck command and the absence of specific security measures.
s4/s4.h	COMPLIANT	The provided code snippet appears to be a header file for a system, and does not contain any information that would indicate a direct violation of HIPAA Business Associate Controls.
s4/s4.c	COMPLIANT	The code snippet appears to be a general-purpose utility file and does not explicitly demonstrate HIPAA-compliant practices, such as data encryption or secure communication.